SE 228: Analysis and Design of Algorithms

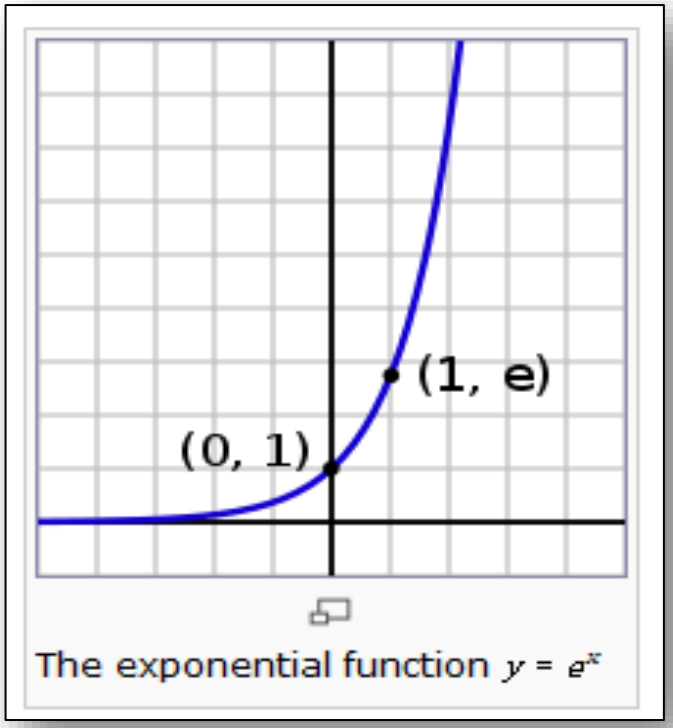
A Short Review for Midterm

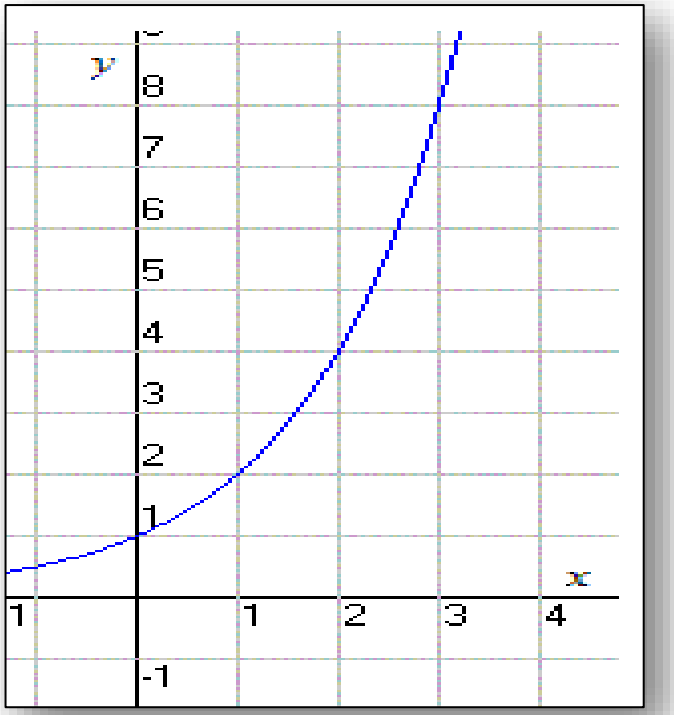
Measuring Efficiency :Types of Mathematical Functions

$y = x^a$: power function

$y = a^x$: exponential function

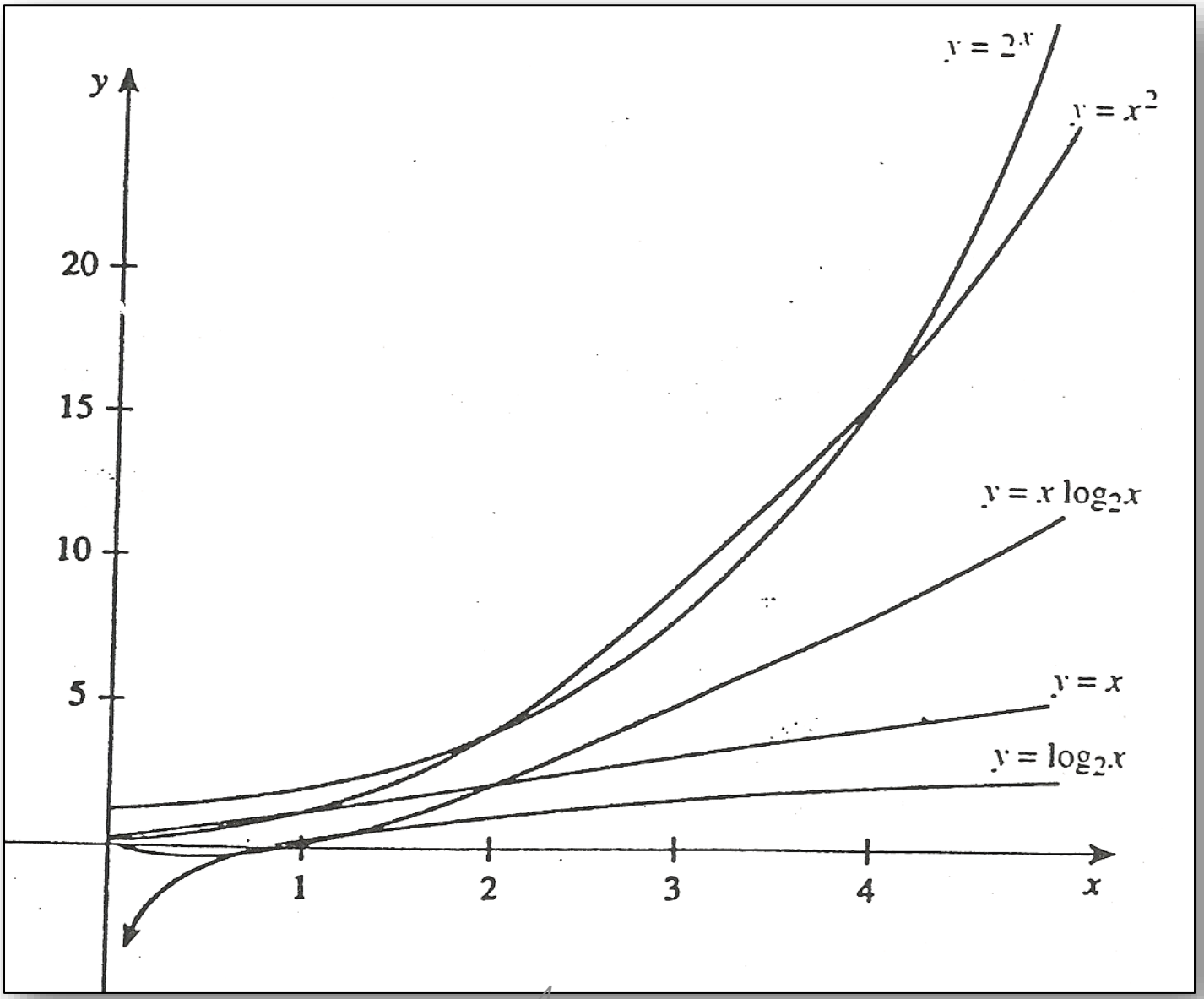
The Exponential Function: $y = b^x$, $b>1$, b - base





$y=2^x$

Comparison of Functions



Comparison of Functions for Increasing n

	lg n	n	n lg n	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

The last line for n= 100 : The value of 2^n is ~ 10^{30}

Assume a function represents the number of operations executed and one operation takes 1 ns. Time needed for input size n:

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3 nsec	0.01 μ	0.02 μ	0.06 μ	0.51 μ	0.26 μ
16	4 nsec	0.02 μ	0.06 μ	0.26 μ	4.10 μ	65.5 μ
32	5 nsec	0.03 μ	0.16 μ	1.02 μ	32.7 μ	4.29 sec
64	6 nsec	0.06 μ	0.38 μ	4.10 μ	262 μ	5.85 cent
128	0.01 μ	0.13 μ	0.90 μ	16.38 μ	0.01 sec	10^{20} cent
256	0.01 μ	0.26 μ	2.05 μ	65.54 μ	0.02 sec	10^{58} cent
512	0.01 μ	0.51 μ	4.61 μ	262.14 μ	0.13 sec	10^{135} cent
2048	0.01 μ	2.05 μ	22.53 μ	0.01 sec	1.07 sec	10^{598} cent
4096	0.01 μ	4.10 μ	49.15 μ	0.02 sec	8.40 sec	10^{1214} cent
8192	0.01 μ	8.19 μ	106.50 μ	0.07 sec	1.15 min	10^{2447} cent
16384	0.01 μ	16.38 μ	229.38 μ	0.27 sec	1.22 hrs	10^{4913} cent
32768	0.02 μ	32.77 μ	491.52 μ	1.07 sec	9.77 hrs	10^{9845} cent
65536	0.02 μ	65.54 μ	1048.6 μ	0.07 min	3.3 days	10^{19709} cent
131072	0.02 μ	131.07 μ	2228.2 μ	0.29 min	26 days	10^{39438} cent
262144	0.02 μ	262.14 μ	4718.6 μ	1.15 min	7 mnths	10^{78894} cent
524288	0.02 μ	524.29 μ	9961.5 μ	4.58 min	4.6 years	10^{157808} cent
1048576	0.02 μ	1048.60 μ	20972 μ	18.3 min	37 years	10^{315634} cent

Table 1.1 Running times for different sizes of input. "nsec" stands for nanoseconds, "μ" is one microsecond and "cent" stands for centuries.

Analysis : Asymptotic Performance

- We are interested in *asymptotic analysis* of algorithms
- *Asymptotic performance*: How does algorithm behave as the problem size gets very large?
    - Running time/Complexity
  - Use the RAM model:
    - All memory equally expensive to access
    - No concurrent operations
    - All reasonable instructions take unit time
      - Except, of course, function calls
    - Constants, low-order terms don't matter

Asymptotic Notation

- We say run time or complexity of algorithm X is $O(n^2)$
  - Read O as "Big-Oh" (you'll also hear it as "order")
- **Upper Bound Notation:**

  f(n) is O(g(n))

  if there exist positive constants *c* and n_0 such that

  $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- Big O fact:
  - A polynomial of degree *k* is $O(n^k)$

Lower Bound Notation

- We say, for example , Insertion Sort's run time is $\Omega(n)$
- In general a function
  - f(n) is $\Omega(g(n))$

    if there exist positive constants *c* and n_0 such that

    $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$
- We say $f(n) = \Theta(g(n))$

  if and only if

  $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$

Big O Examples

Example 1: Prove that running time $T(n) = n^3 + 20n + 1$ is $O(n^3)$

Proof: by the Big-Oh definition, $T(n)$ is $O(n^3)$ if $T(n) \leq c \cdot n^3$ for some $n \geq n_0$. Let us check this condition: if $n^3 + 20n + 1 \leq c \cdot n^3$ then $1 + \frac{20}{n^2} + \frac{1}{n^3} \leq c$. Therefore, the Big-Oh condition holds for $n \geq n_0 = 1$ and $c \geq 22$ $(= 1 + 20 + 1)$. Larger values of n_0 result in smaller factors c (e.g., for $n_0 = 10$ $c \geq 1.201$ and so on) but in any case the above statement is valid.

Another solution : *(c-1) $n^3 \geq 20n+1$,Take c=2. $n \geq 5$ satisfies the inequality.*

More Big O Examples

$13n^3 = O(n^3)$

$3n^3 + 18 = O(n^3)$

$18n^2 + 10n * \log(n) + 100n + 10^{20} = O(n^2)$

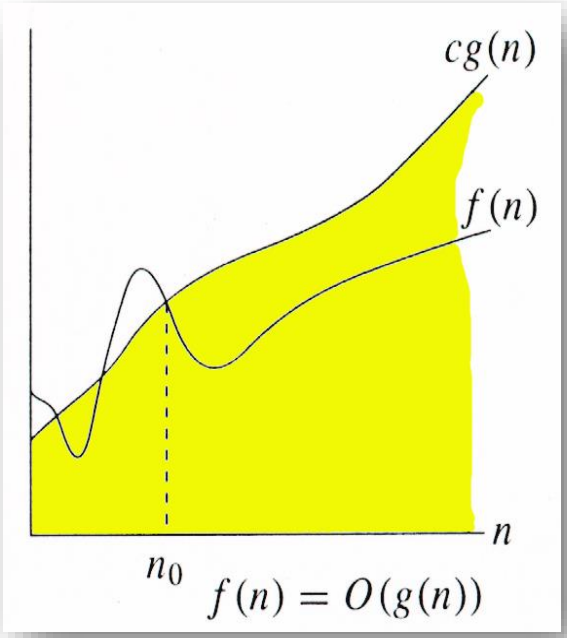
$3\log(n) + 2n^{1/2} = O(n^{1/2})$

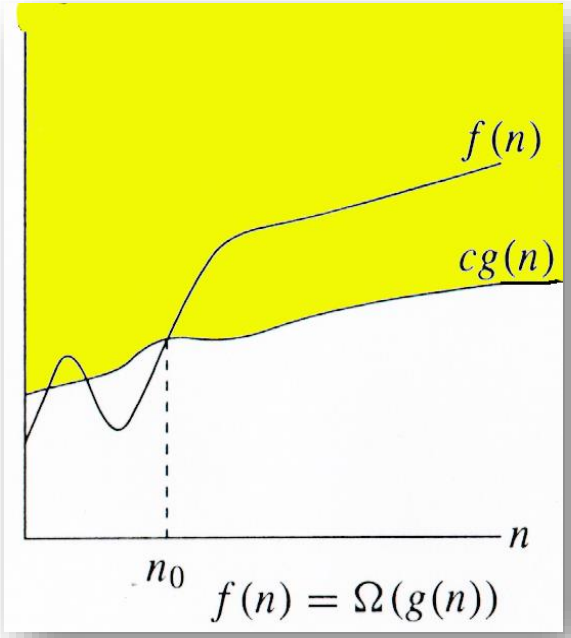
$2^{100} = O(1)$ (Because complexity does not depend on n)

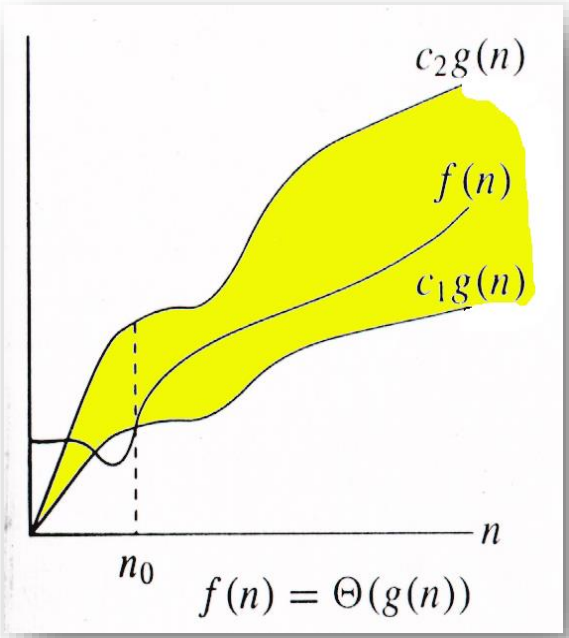
$T_{linearSearch}(n) = O(n)$

$T_{binarySearch}(n) = O(\log(n))$

Relations Between, *O*, Ω, Θ







Big O *is similar to less than or equal, an* ***upper bound*****.**

Big Omega *is similar to greater than or equal,* ***a lower bound.***

Ranking of Algorithmic Behaviors

Function	Common Name
$N!$	factorial
2^N	Exponential
N^d, $d > 3$	Polynomial
N^3	Cubic
N^2	Quadratic
$N \sqrt{N}$	n root n
$N \log N$	nlogn
N	Linear
$\sqrt{N}$	Root - n
$\log N$	Logarithmic
1	Constant

Loops and Complexities

- Suppose we have a statement “c++”
- For each pseudocode snippet, determine how many times the line “count++” will run as a function of N.

1.for (i = 1; i <= N; i++)

for (j = 1; j <= 2 * N; j++)

c++;

→$T(N) = 2N^2 = O(N^2)$

2. for (i = 1; i <= N; i++) c++;

for (i = 1; i <= 2 * N; i++) c++;

→T(N) = 2N+N or 3N. This is O(N).

Loops and Complexities

```
3. for (i = 1; i <= N; i++)
        for (j = 1; j <= 2 * N; j++)
                count++;
   for (i = 1; i <= 2 * N; i++)
        count++;
```

- The first double for loop runs $2N^2$ times . The second loop runs 2N times. So,

  $T(N)= 2N^2 + 2N$ or $2N(N+1)$.

  This is $O(N^2)$.

Recurrence Relations

The concept: Recurrence relations are recursive definitions of mathematical functions or sequences. For example, the recurrence relation

$f(n) = f(n-1) + 2n - 1$

$f(0) = 0$

defines the function $f(n) = n^2$. (Substitute n=1,n=2,….to discover the general term)

The recurrence relation

$f(n) = f(n-1) + f(n-2)$

$f(1) = 1$

$f(0) = 1$

Defines the Fibanocci sequence 1,1,2,3,5,8,13,....

Solving Recurrences

- The “iteration method”
  - Expand the recurrence
  - Work some algebra to express as a summation
  - Evaluate the summation
  - Try to discover a pattern during evaluation.

Solving Recurrences:Example

$T(n) = T(n-1) + O(n)$, $T(1)=1$

$= (T(n-2) + O(n-1)) + O(n)$

$= T(n-2) + O(n-1) + O(n)$

$= T(n-3) + O(n-2) + O(n-1) + O(n)$

...

$= T(1) + O(2) + ... + O(n-1) + O(n)$

$= O(1 + 2 + ... + n-1 + n)$

$= O(n*(n+1)/2)$

$= O(n^2)$

Algorithm Design Methods

- Brute Force
- Divide and Conquer
- Greedy
- Dynamic Programming
- ……

Brute Force

- A straightforward approach to solving a problem, usually based on problem statement and definitions of the concepts involved
- "Force" comes from using computing power not intellectual power
- In short, "brute force" means "Just do it!"
- Wide applicability due to simplicity

Examples

- Simple sorts : Selection sort, buble sort…
- Sequential search
- Matrix multiplication
- Finding closest pairs
- Exhaustive searches :TSP
- ……..

Brute Force

However

- It rarely yields efficient algorithms.
- Some brute force algorithms are unacceptably slow.
- It is neither as constructive nor creative as some other design techniques.

Divide and Conquer

- *Divide-and-conquer* method for algorithm design:
  - If the problem size is small enough to solve it in a straightforward manner, solve it. Else:
    - **Divide**: Divide the problem into two or more disjoint smaller *subproblems*
    - **Conquer**: Use divide-and-conquer recursively to solve the subproblems
    - **Combine**: Take the solutions to the subproblems and combine these solutions into a solution for the original problem

Divide and Conquer : Examples

- Coin changing
- Mergesort
- Quicksort
- Binary search
- Tree traversals
- Closest pair of points
- ……

Sorting :Merge Sort

- Merge sort:
  - Divide-and-conquer:
    - Split array in half
    - Recursively sort subarrays
    - Merge subarrays . Linear-time merge step
  - O(n log n) complexity : All cases are the same!
  - But,doesn’t sort in place, uses additional storage during merges.

Analysis of Merge Sort

Statement	Effort
`MergeSort(A, left, right) {`	`T(n)`
`  if (left < right) {`	`Θ(1)`
`    mid = floor((left + right) / 2);`	`Θ(1)`
`    MergeSort(A, left, mid);`	`T(n/2)`
`    MergeSort(A, mid+1, right);`	`T(n/2)`
`    Merge(A, left, mid, right);`	`Θ(n)`
`  }`	
`}`	

So T(n) = $\Theta(1)$ when n = 1, and

$2T(n/2) + \Theta(n)$ when n > 1

Solving this recurrence gives T(n) = n log n

Sorting : Quicksort

- Quick sort:
  - Divide-and-conquer:
    - Partition array into two subarrays, recursively sort subarrays
    - All values of first subarray < all values of second subarray
    - **No merge step needed!**
  - O(n log n) best and average case
  - But, $O(n^2)$ worst case
  - Fast in practice
    - Naïve implementation: worst case on nearly sorted input

Analyzing Quicksort

- What will be the worst case for the algorithm?
  - Partition is always maximally unbalanced : 0 and n-1
- What will be the best case for the algorithm?
  - Partition is nearly perfectly balanced
- Which case is more likely in practice?
  - The latter,
  - *What is the particular type of input composition for the worst case?*

  Already-sorted input

Analyzing Quicksort

- In the worst case:

  $T(1) = \Theta(1)$

  $T(n) = T(n - 1) + \Theta(n)$

- Works out to

  $T(n) = \Theta(n^2)$

Analyzing Quicksort

- In the best case:

  $T(n) = 2T(n/2) + \Theta(n)$

- Works out to

  $T(n) = \Theta(n \log n)$

- Average case also works out to

  $T(n) = \Theta(k\ n \log n)$

  $= \Theta(\ n \log n)$

Reminder :Graphs

- A graph G = (V, E)
  - V = set of vertices
  - E = set of edges
    = subset of (|V|*|V|)

  Thus |E| = O(|V|^2)
  - Directed or undirected
- Representation methods:
  - Adjacency list
  - Adjacency matrix
- Traversal(Search) methods
  - Breadth-first search
  - Depth-first search

Greedy Algorithms

- A *greedy algorithm* always makes the choice that looks best at the moment.
- Indicators:
  - Optimal substructure:

  If optimal solution of the given problem can be obtained by using optimal solutions of its subproblems.
  - Greedy choice property:

  A locally optimal choice leads to a globally optimal solution
- Example problems:
  - Activity selection: Set of activities, with start and end times. Optimize the set of activities.
  - Shortest Path , MST…

Single-Source Shortest Paths

- Problem: given a **weighted directed** graph G, find the minimum-weight path from a given source vertex s to another vertex v
  - "Shortest-path" = minimum weight
  - Weight of path is sum of edge weights
  - E.g., a road map: what is the shortest path from Bornova to Konak ?
  - Can be extended to : From one source to all other vertices

Single-Source Shortest Paths

- Greedy Solution : Djkstra's algorithm
- The algorithm generates a *shortest path tree* with given source as the root. We maintain two sets:
  - First set contains vertices included in shortest path tree,
  - Second set includes vertices not yet included in shortest path tree.
  - At every step of the algorithm, we find a vertex in the second set which has a minimum distance from the source and move it to the first set

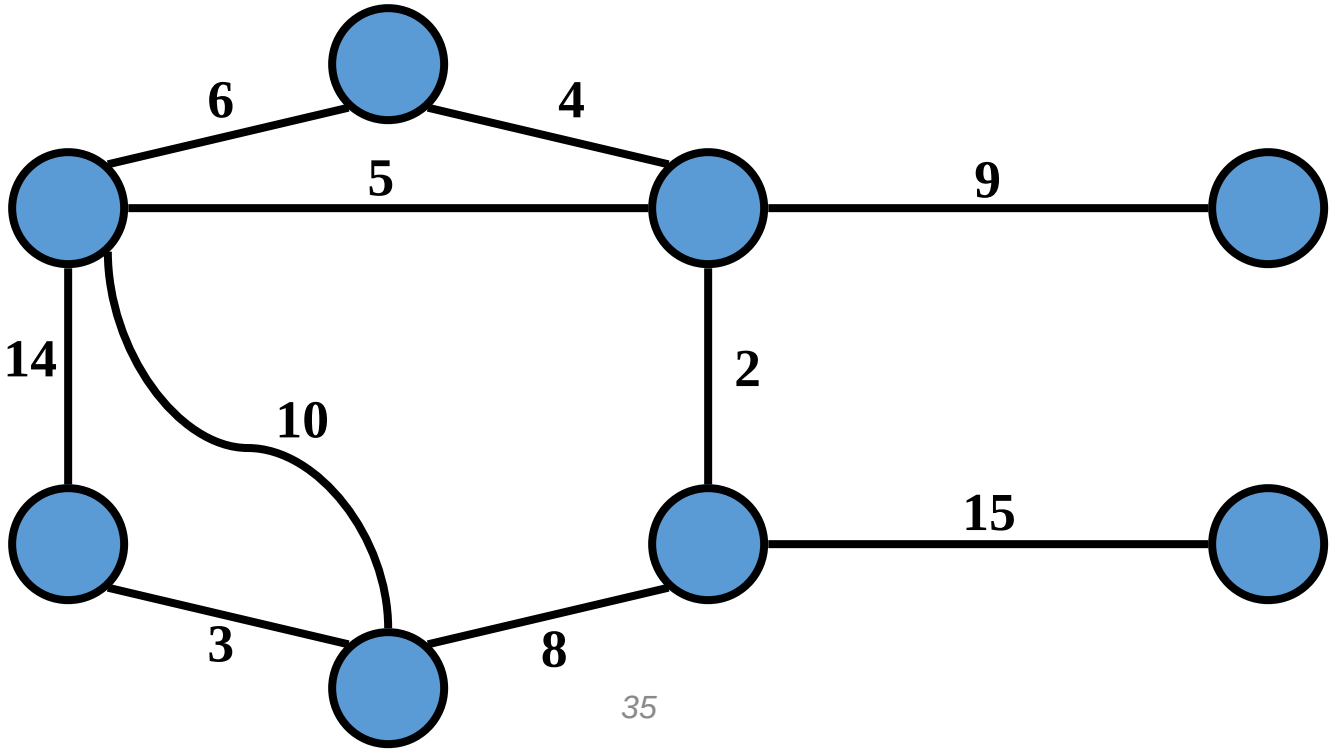
Time Complexity

- The simplest implementation of the Dijkstra's algorithm stores vertices in an adjacency list:
- Assume |V| vertices and |E| edges
- Initialization O(|V|)
- While loop in the algorithm : O(|V|)
  - Find and remove min distance vertices O(|V|)
- Potentially |E| updates
    - Update costs O(1)

Total time $O(|V^2| + |E|) = O(|V^2|)$

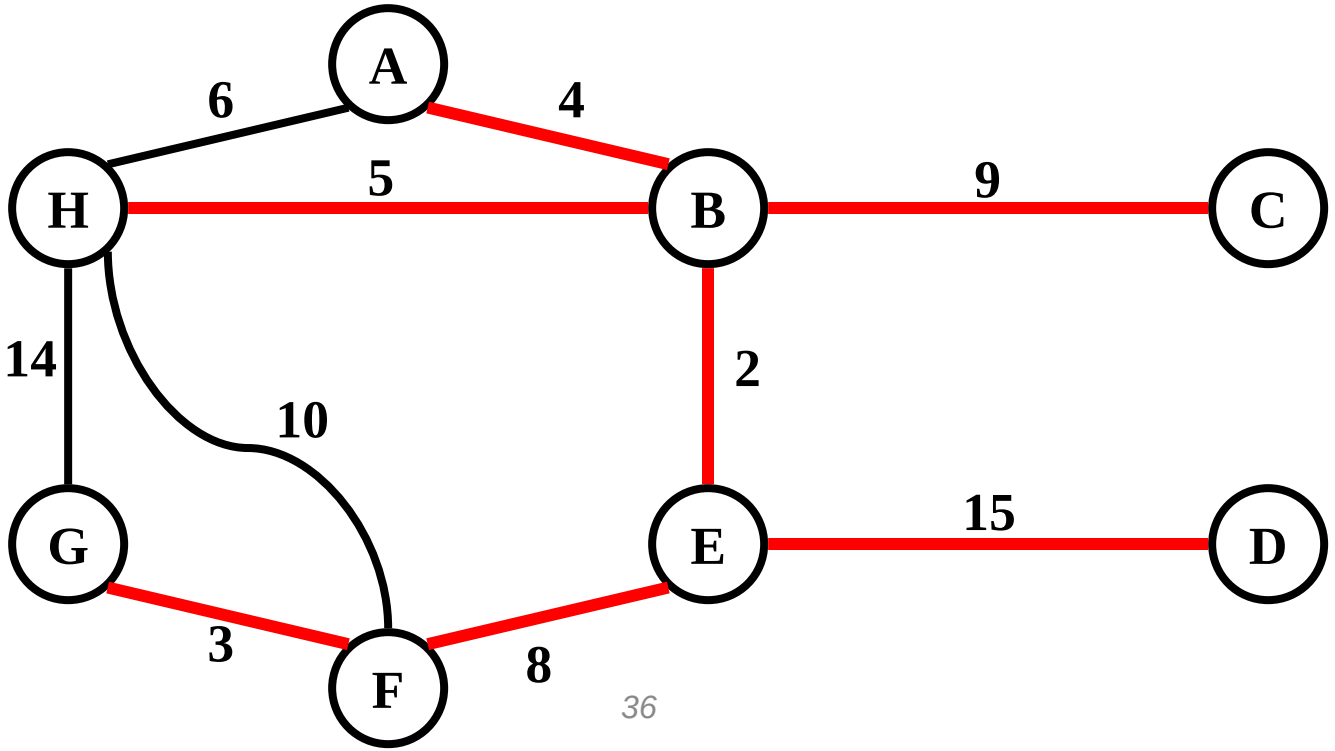
Minimum Spanning Tree

- Problem: given a **connected, undirected, weighted** graph, find a *spanning tree* using edges that minimizes the total weight. Example :



Minimum Spanning Tree

- What is the MST ?
- Answer: Red path,total weight = 46. (May not be unique!)



MST Algorithms

- Two well known algorithms :
  - Kruskal's algorithm
  - Prim's algorithm

Kruskal's MST Algorithm

Informal description:

- Consider edges in increasing order of weight. Initially, each vertex is in its own MST.
- Merge two nodes(MSTs) that have the shortest edge between them.
- Merging is not allowed to cause a cycle at any stage.
- Always try to add cheapest available edge that will not violate the tree property

  →locally optimal greedy choices lead to optimal solution.

  (Study the example in lecture slides!)

Prim's Algorithm

- Prim's algorithm for finding the MST:
  - Start with an arbitrary vertex to form a MST on one vertex
  - At each step, add vertex *v* not yet in the MST that has an edge with least weight that connects *v* to the existing minimum spanning sub-tree
  - Continue until we have *n* – 1 edges and *n* vertices in the MST
- How to continue ? For each vertex not yet in the tree, keep track of its cheapest edge to the tree constructed so far.
- To find min weight outgoing edge, find minimum among these values.

  (Study the example in lecture slides!)

Kruskal and Prim : Difference

- **Kruskal**'s builds a minimum spanning tree by adding one **edge** at a time. The next line is always the shortest (minimum weight) ONLY if it does NOT create a cycle.
- **Prim**'s builds a mimimum spanning tree by adding one **vertex** at a time. The next vertex to be added is always the one nearest to a vertex already on the graph.
- If the MST is unique,both algorithms produce the same MST!